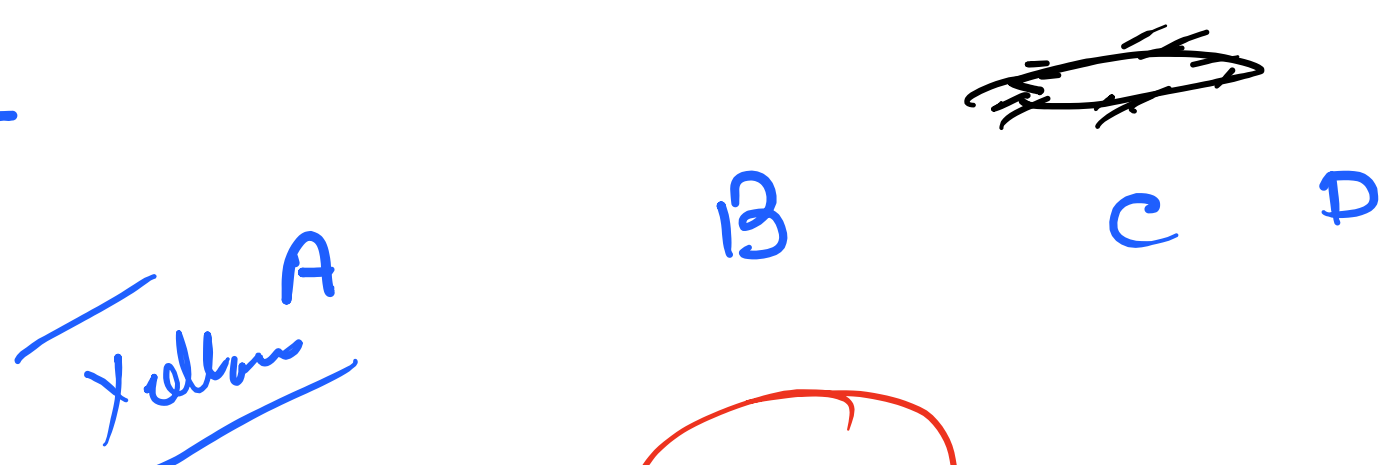
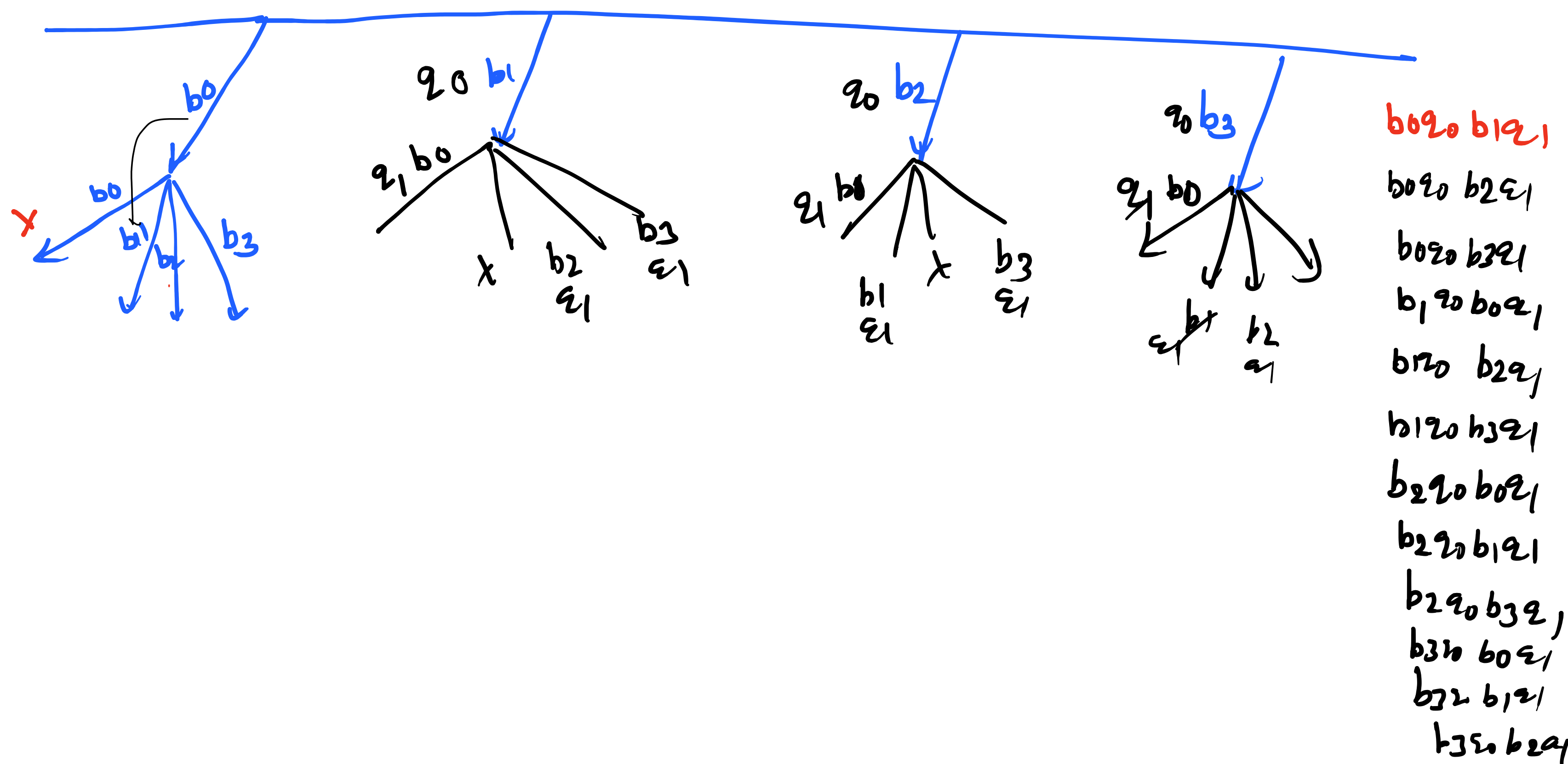
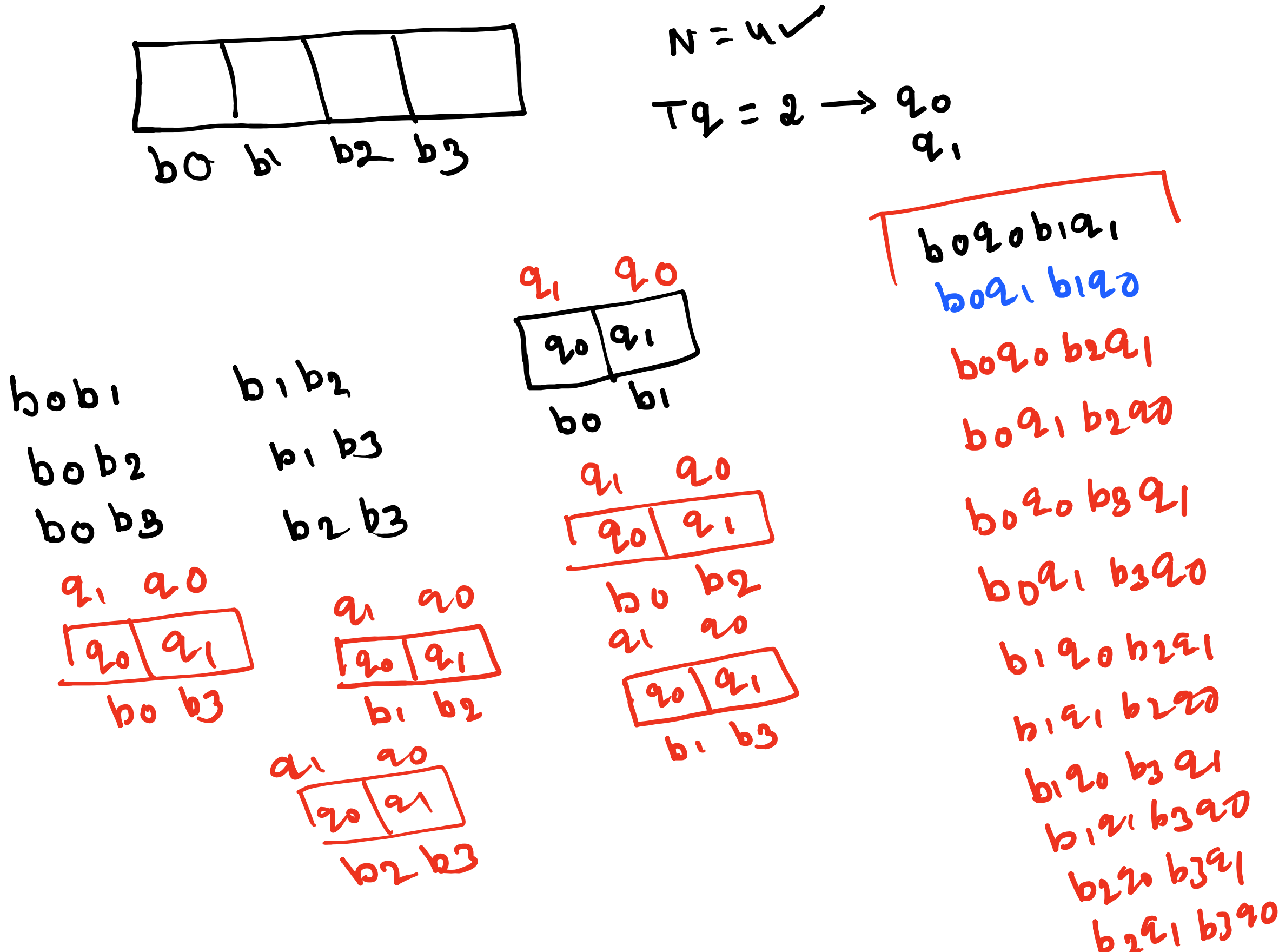


① Backtracking

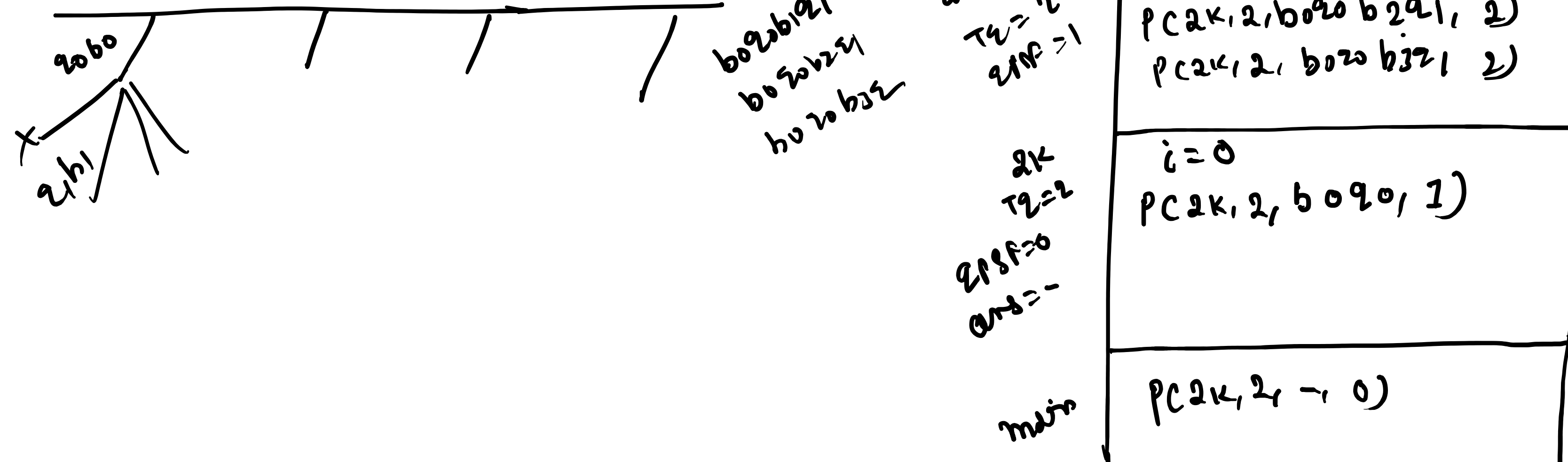


$$N_P_r = \frac{n!}{n-r!}$$
$$N_C_r = \frac{n!}{(n-r)! r!}$$

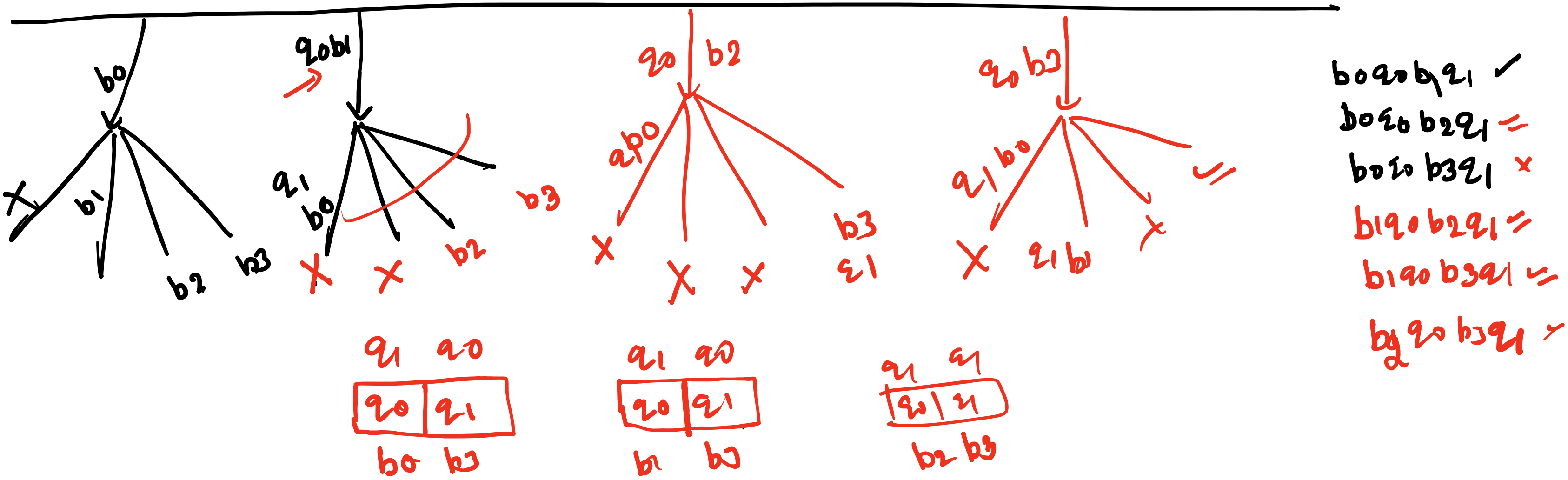
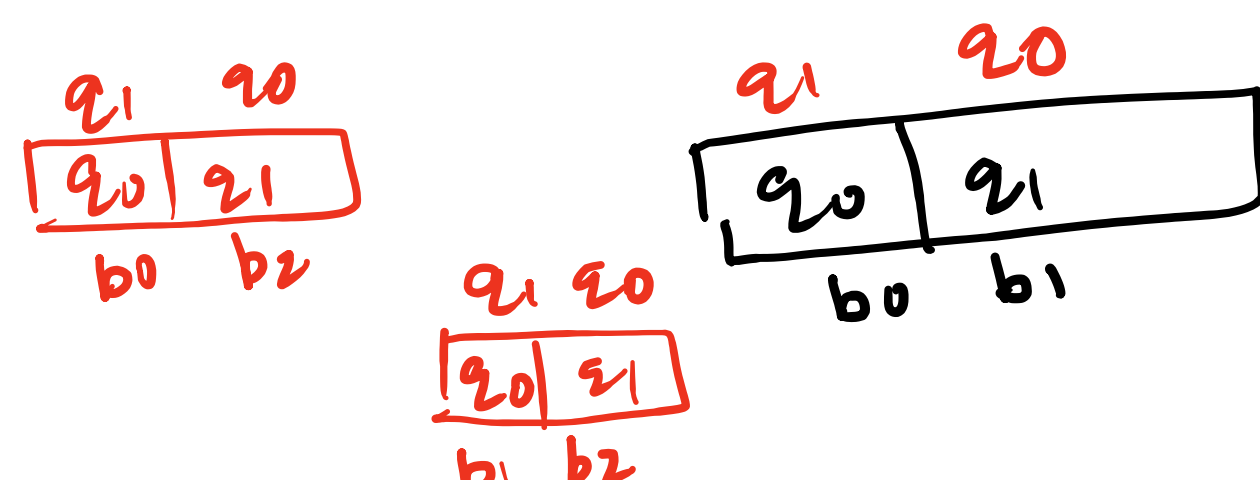
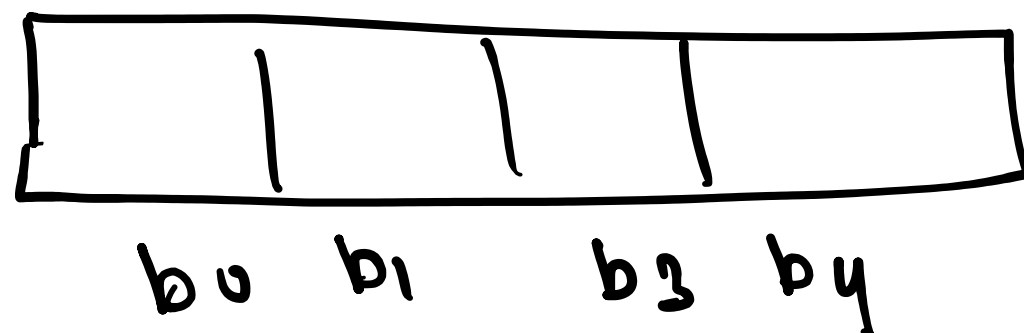
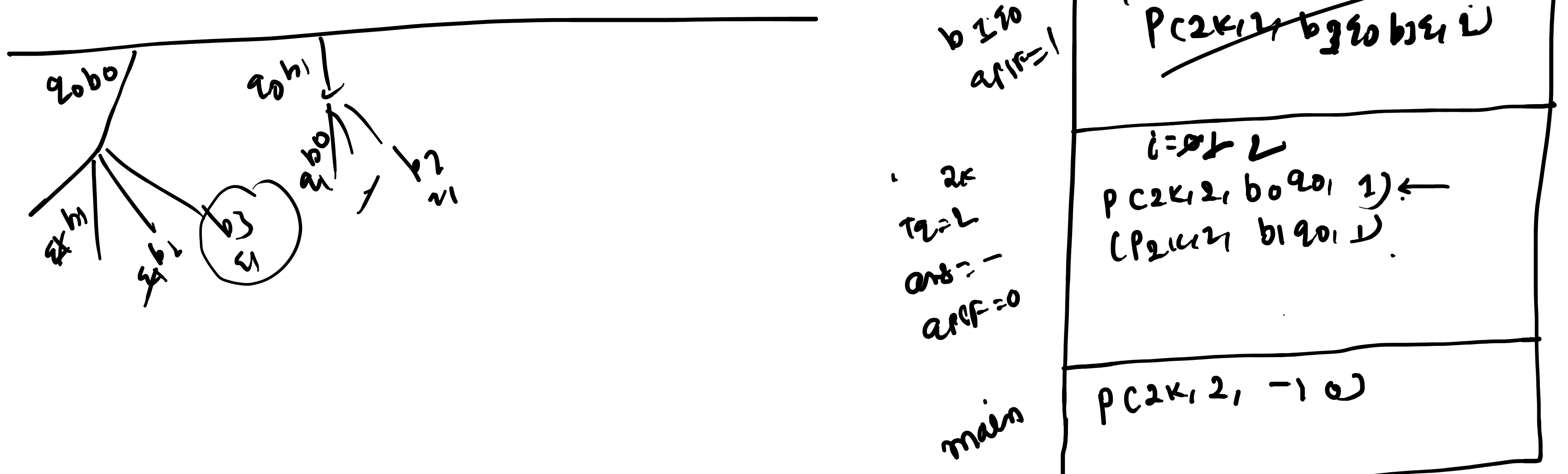
$$4C_2 = 6$$



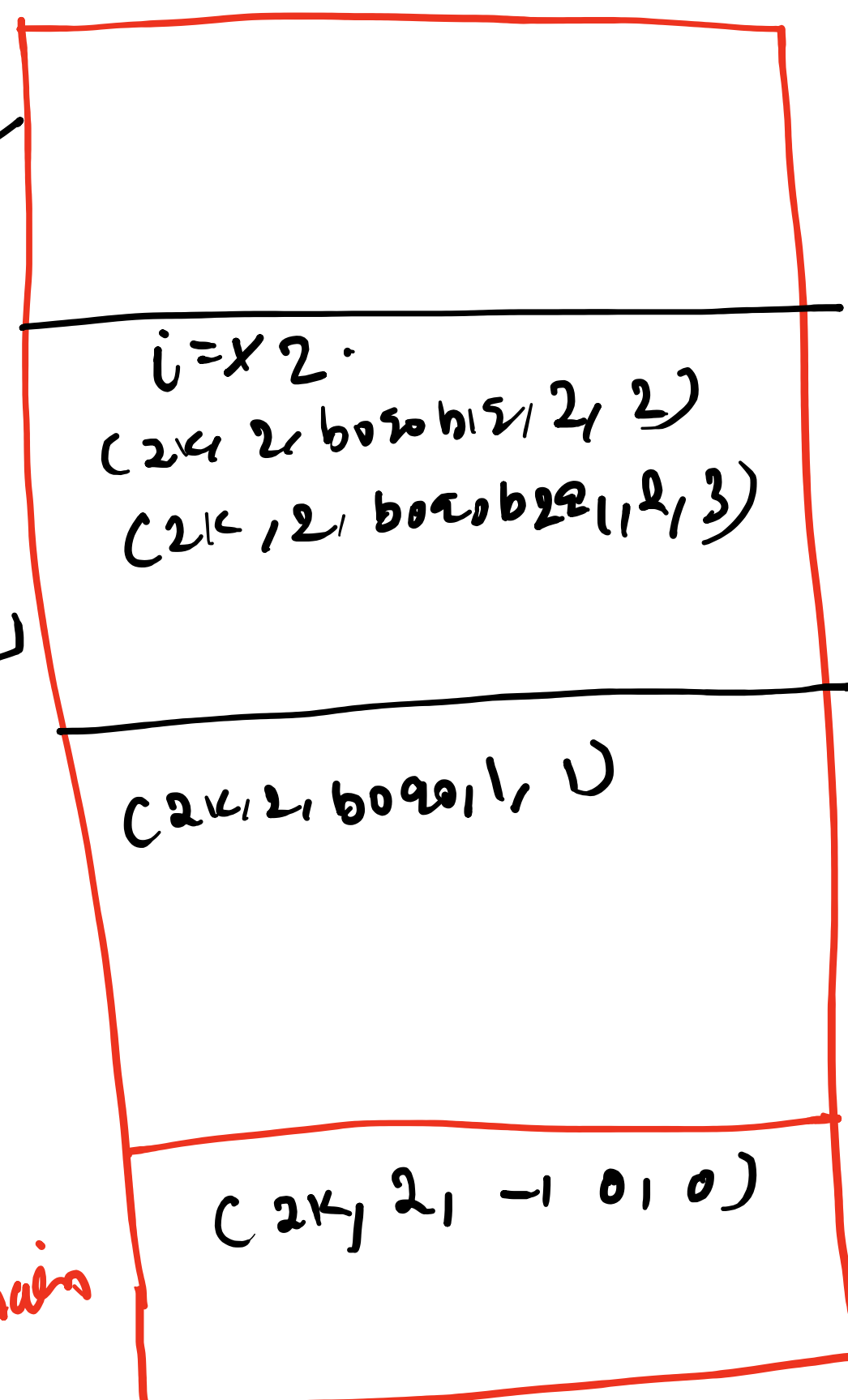
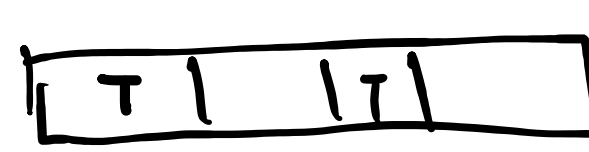
```
public static void Permutation(boolean[] board, int tq, String ans, int qpsf) {  
    if(qpsf==tq) {  
        System.out.println(ans);  
        return;  
    }  
    for (int i = 0; i < board.length; i++) {  
        if(board[i]==false) {  
            board[i]=true;  
            Permutation(board, tq, ans+"b"+i+"q"+qpsf, qpsf+1);  
        }  
    }  
}
```



```
public static void Permutation(boolean[] board, int tq, String ans, int qpsf) {  
    if(qpsf==tq) {  
        System.out.println(ans);  
        return;  
    }  
    for (int i = 0; i < board.length; i++) {  
        if(board[i]==false) {  
            board[i]=true;  
            Permutation(board, tq, ans+"b"+i+"q"+qpsf, qpsf+1);  
            board[i]=false; //UN-DO  
        }  
    }  
}
```



```
public static void Combination(boolean[] board, int tq, String ans, int qpsf, int idx) {  
    if(qpsf==tq) {  
        System.out.println(ans);  
        return;  
    }  
    for (int i = idx; i < board.length; i++) {  
        if(board[i]==false) {  
            board[i]=true;  
            Combination(board, tq, ans+"b"+i+"q"+qpsf, qpsf+1, i+1);  
            board[i]=false; //UN-DO  
        }  
    }  
}
```



$ans += b + i + q$

